

Module 4

UART — UVM+SV Verification

Learning objectives

- UART UVM agent: Transaction (UartTxTransaction), sequence (UartTxSequence), driver (UartTxDriver),...
- Interface: uart_tx_if (clk, rst_n, start, data, tx, baud_tick) connects UVM to DUT.
- Loopback check: TX path (driver → DUT → monitor → scoreboard) and RX path (DUT rx_valid/rx_data → s...

Prerequisites

- Module 1 (spec → RTL), Module 2 (UVM+SV, uvm_smoke), Module 3 (UART protocol + RTL + basic TB).
- Verilator, Make, C++ compiler, UVM (UVM_HOME or vendored UVM).

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["UART protocol recap (8N1)"]

T2["Toolchain"]

T1 --> T2

T2 --> N["SPI"]

Extend UART verification to ****UVM+SV**** — UART agent (transaction, sequence, driver, monitor, scoreboard); run on Verilator.

Overview

- Module 4 builds on Module 3 (UART protocol + RTL + basic testbench):

How to learn this module

Suggested learning path

- Read the detailed UART learning guide:
UART_LEARNING_GUIDE.md — what UART is, how it works
(frame,...
- Read the protocols + UVM overview:
LEARNING_GUIDE_PROTOCOLS_AND_UVM.md — Part B § 5.
How UVM Maps t...

Design architecture

RTL / block architecture

Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart TB

subgraph dut["UART DUT (same as Module 3)"]

BG["baud_gen"]

TX["uart_tx"]

RX["uart_rx"]

BG --> TX

Module 4: DUT / block hierarchy (see module4/examples/).

Verification environment architecture

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart TB

SEQ["UartTxSequence"] --> DRV["UartTxDriver"]

DRV --> DUT["uart_tx / uart_rx"]

DUT --> MON["UartTxMonitor\nreconstruct from tx"]

MON --> SB["UartTxScoreboard"]

DUT -->|loopback rx_valid| SB

Module 4: UVM layers, agents, and checkers for this phase.

1. UART DUT (reused from Module 3)

- dut/: ``uart_tx``, ``uart_rx``, ``baud_gen`` — identical RTL to ``uart_baseline``.
- Loopback in top: ``rx = tx`` for closed-loop checking.
- Why reuse: Upgrade verification (UVM) without touching proven RTL.

2. UVM testbench hierarchy

- uart_tx_if: virtual interface for driver/monitor.
- UartTxTransaction / Sequence / Driver / Monitor /
Scoreboard / Agent / Env / Test — same pattern as...
- Monitor reconstructs bytes from the serial `tx` line
using UART 8N1 rules.

3. Dual check paths

- TX path: sequence → driver → DUT → monitor → scoreboard (`observed\_tx`).
- RX path: loopback into `uart\_rx`; `check\_rx\_byte()` when `data\_valid` asserts.

Verification & testing methods

Testing and closure flow

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TB

SEQ["UartTxSequence"] --> DRV["Driver"]

DRV --> DUT["UART DUT"]

DUT --> MON["Monitor on tx"]

MON --> SB["Scoreboard TX"]

DUT --> RX["RX hook"]

Module 4: how tests, checks, and metrics close verification goals.

1. Directed UART regression

- Sequence sends 0x00, 0x01, 0x55, 0xAA, 0xFF — repeatable regression before randomization.

2. Scoreboard strategy

- Expected queue loaded in test; compare to monitor-reconstructed TX and to RX hook data.
- Fail fast on mismatch; read UVM SCOREBOARD summary at end of run.

3. What to add next

- Random baud, framing-error injection, coverage on ``busy`` / ``data_valid``.
- See [LEARNING_GUIDE_PROTOCOLS_AND_UVM.md § 6](LEARNING_GUIDE_PROTOCOLS_AND_UVM.md#6-uart-uvm-mappin...)

Syllabus topics

1. UART protocol recap (8N1)

- One byte = start (0) + 8 data (LSB first) + stop (1); ``baud_tick`` times each bit.
- Driver/monitor must follow the same rules as the RTL.

2. Toolchain

- Verilator ``-sv``, ``--timing``, UVM includes; ``make`
SIM=verilator TEST=test_uart_uvm`; ``+UVM_TESTNAME=...`

Hands-on examples

Module 4 self-check

```
$ ./scripts/module4.sh --help
```

Terminal: module4_check

```
[[0;36m===== [[0m
[[0;36mModule 4: Demo commands (UART UVM) [[0m
[[0;36m===== [[0m

From repo root:

# 1) Environment sanity:
verilator --version
make --version
echo "$UVM_HOME"

# 2) Run the UART UVM example (inside the example directory):
cd module4/examples/uart_uvm
make SIM=verilator TEST=test_uart_uvm

# 3) Inspect logs and waveforms:
ls obj_dir
# If a VCD is produced:
ls *.vcd

[[1;33m[INFO] [[0m See module4/EXAMPLES.md and module4/examples/uart_uvm/README.md for more.
```

Exercise scaffold

```
./scripts/module4.sh --scaffold
```

Example 1: UART UVM

- UVM phases and reporting (DRIVER, MONITOR, SCOREBOARD).
- TX and RX scoreboard checks (TX: monitor observes serial line; RX: loopback byte checked via scoreb...
- Final scoreboard summary (TX/RX matches and mismatches).

Demo: UART UVM

```
$ ./scripts/module4.sh --run
```

Terminal: uart_uvm

```
[0:36m=====
[0:36mModule 4: Run uart_uvm example (logging to /mnt/d/proj/universal-verification-methodology/lea
[0:36m=====

[1:33m[INFO][0m Setting UVM_HOME to: /mnt/d/proj/universal-verification-methodology/learn_uart_spi
[Thu May 28 15:27:14 CST 2026] Running make SIM=verilator TEST=test_uart_uvm
Compiling UART UVM test with Verilator...
verilator -sv --timing --trace -Wno-fatal -Wno-TIMESCALEMOD -Wno-WIDTHTRUNC -I/mnt/d/proj/universal-
--binary \
/mnt/d/proj/universal-verification-methodology/learn_uart_spi_i2c/tools/uvm-2017/1800.2-2017-1.0/sr
dut/*.v test_uart_uvm.sv \
--top-module test_uart_uvm
/bin/sh: 1: verilator: not found
make: *** [Makefile:40: compile] Error 127
```

Practice & assessment

What you should know

- Describe the UART UVM agent (transaction, sequence, driver, monitor, scoreboard).
- Explain loopback (TX→RX) and how TX and RX are both checked in the scoreboard.
- Run the `uart_uvm` example and interpret UVM output.
- Ready for Module 5: SPI protocol + RTL + basic testbench.

Exercises

- Run uart_uvm
- Trace one transaction
- Change the sequence
- Optional: Loopback path

Assessment checklist

- Can describe the UART UVM agent: transaction (one byte), sequence, driver, monitor, scoreboard.
- Can explain loopback and how TX and RX are both checked.
- Can run `uart_uvm` and interpret UVM output (phases, scoreboard summary).
- Ready for Module 5: SPI protocol + RTL + basic testbench.

Summary & next steps

- Extend UART verification to ****UVM+SV**** — UART agent (transaction, sequence, driver, monitor, scoreb...
- Complete module4/CHECKLIST.md
- Review module4/EXAMPLES.md and run each lab
- Next: SPI